Obradoiro Git e GitHub · Instrucións do Usuario 1

Ti es o **Usuario 1**: creas o repositorio e desenvolves as funcións de estatística e o resumo do informe. O teu compañeiro ten outras instrucións. Cando vexas 🤝, toca coordinarse con el.

Que necesitas: conta de GitHub, Git, Python 3, un editor e o ficheiro `proxecto-notas.zip`.

Comprobar sempre: `python probas.py` indica se cada función está OK, PENDENTE ou FALLA. Execútao antes de cada commit, sempre desde a carpeta raíz do proxecto (non desde `src`).

Fase 0 · Crear o repositorio

1. Configura Git (só a primeira vez):

```
git config --global user.name "O teu nome"
git config --global user.email "o-teu-email@exemplo.com"
git config --global pull.rebase false
ssh -T git@github.com    # comproba a chave SSH: debe saudarte co teu usuario
```

2. En GitHub, preme **New repository**. Nome: `proxecto-notas`. **Non** marques README, .gitignore nin licenza: o repositorio debe crearse baleiro.

3. Copia a URL (botón verde **Code**, pestana **SSH**) e clona o repositorio:

```
git clone git@github.com:O-TEU-USUARIO/proxecto-notas.git
cd proxecto-notas
```

4. Descomprime `proxecto-notas.zip` e copia **o contido** da carpeta dentro da carpeta clonada, incluído o ficheiro oculto `.gitignore`. Debes ter as carpetas `datos` e `src` e, na raíz, `main.py` e `probas.py`.

5. Executa `python probas.py`: todo debe aparecer como PENDENTE.

6. Garda e sube o esqueleto:

```
git add .
git commit -m "Esqueleto inicial do proxecto"
git branch -M main
git push -u origin main
```

7. Recarga a páxina do repositorio en GitHub e comproba que aparecen os ficheiros.

8. Vai a **Settings** → **Collaborators** → **Add people** e invita ao teu compañeiro polo seu nome de usuario.

> 🤝 **Avisa ao teu compañeiro** de que xa pode clonar o repositorio. Cando o faga, executade os dous `git log --oneline`: debedes ver o mesmo commit.

Fase 1 · O teu ficheiro: `estatisticas.py`

1. Crea a túa rama:

```
git checkout main
git pull
git checkout -b fase1-estatisticas
```

2. Completa `media()`, `nota_maxima()` e `nota_minima()`. Substitúe cada `raise NotImplementedError` polo teu código.

3. Executa `python probas.py` ata que o bloque "FASE 1 · Usuario 1" estea todo en OK.

4. Garda o traballo, actualiza a rama con `main` e súbea. Nesta fase o normal é que Git responda *Already up to date* ou faga un merge sen conflitos, porque cada un traballa nun ficheiro distinto. Aínda así, faino sempre:

```
git add src/estatisticas.py
git commit -m "Implementa as funcións de estatísticas"
git pull origin main
git push -u origin fase1-estatisticas
```

5. En GitHub preme **Compare & pull request**. Comproba: base `main` ← compare `fase1-estatisticas`. Escribe un título e unha descrición, escolle o teu compañeiro en **Reviewers** e preme **Create pull request**.

> 🤝 **Avisa ao teu compañeiro** de que o teu PR está listo. Cando el abra o seu, revísao ti.

6. Abre o PR do compañeiro, pestana **Files changed**. Deixa polo menos un comentario nunha liña (botón **+** ao pasar o rato).
7. Preme **Review changes** → **Approve** → **Submit review** e despois **Merge pull request** → **Confirm merge**.

> 🤝 **Cando os dous PR estean combinados**, actualiza o teu `main`:
>
> ```
> git checkout main
> git pull
> python probas.py
> ```
>
> Toda a fase 1 debe estar en OK. Por que non houbo ningún conflito?

Fase 2 · A túa función: `seccion_resumo()`

Nesta fase traballades os dous no mesmo ficheiro, `src/informe.py`.

1. Crea a túa rama partindo do `main` actualizado:

```
git checkout main
git pull
git checkout -b fase2-resumo
```

2. Completa `seccion_resumo()` seguindo o seu docstring.

3. En `informe_completo()`, xusto debaixo do comentario, engade esta liña:

```
partes.append(seccion_resumo(alumnado))
```

4. `python probas.py`: "FASE 2 · Usuario 1" debe estar en OK.

5. Garda, actualiza con `main`, sube e abre o pull request co teu compañeiro como revisor:

```
git add src/informe.py
git commit -m "Engade a sección de resumo ao informe"
git pull origin main
git push -u origin fase2-resumo
```

> 🤝 **Agarda.** O teu compañeiro revisará e combinará o teu PR. Despois terá que resolver un conflito na súa rama. Cando remate, abrirá o seu PR e avisarate.

6. Revisa o PR do teu compañeiro en **Files changed**: verás a súa función e as dúas liñas en `informe_completo()`.
7. Aproba e fai o merge.

> 🤝 **Final.** Executade os dous:
>
> ```
> git checkout main
> git pull
> python probas.py
> python main.py
> ```
>
> Todo debe estar en OK e o informe completo debe aparecer en pantalla. Mirade o historial con `git log --oneline --graph`.

Comandos que vas usar

Comando	Para que serve
`git status`	Ver en que rama estás e que cambiou
`git checkout -b nome`	Crear unha rama e cambiar a ela
`git add` / `git commit -m "..."`	Gardar cambios
`git push -u origin nome`	Subir unha rama nova
`git pull`	Traer os cambios de GitHub

Regra de ouro: nunca traballes en `main`, fai commit antes de cambiar de rama e `git pull origin main` antes de cada push.